# Experiment No. 10

**Title**

Hosting Website Using Git and GitHub Pages

**Aim**

To deploy and host a static website using Git and GitHub Pages for public accessibility and version control integration.

**Software Required**

- Git
- GitHub account
- Text Editor (VS Code / Notepad)
- Web Browser (Chrome / Firefox)

**Theory**

Git is a distributed version control system used to track changes in source code and manage different versions of a project. It allows developers to efficiently handle updates, collaborate, and maintain project history.

GitHub is a cloud-based platform that hosts Git repositories and provides tools for version control, collaboration, and deployment. One of its important features is **GitHub Pages**, which allows users to host static websites directly from a repository.

A static website consists of HTML, CSS, and JavaScript files that do not require server-side processing. GitHub Pages publishes these files as a live website accessible through a public URL.

By using Git commands, users can upload their website files to GitHub, manage updates, and redeploy changes easily. This makes GitHub Pages a simple and effective solution for hosting static websites.

**Procedure (Detailed Steps)**

**Step 1: Install Git**

Download and install Git on your system.
Verify installation using:

```
git --version
```

**Step 2: Create Project Folder**

Create a folder (e.g., my-website) and add your website files:

- index.html
- style.css
- script.js

**Step 3: Open Terminal**

Open Command Prompt or Git Bash inside the project folder.

**Step 4: Initialize Git Repository**

```
git init
```

This initializes a new Git repository in the folder.

**Step 5: Add Files to Staging Area**

```
git add .
```

This command adds all files to the staging area.

**Step 6: Commit the Files**

```
git commit -m "Initial commit"
```

This saves the current version of the project.

**Step 7: Create GitHub Repository**

Go to GitHub and:

- Click on **New Repository**
- Enter repository name (e.g., my-website)
- Click **Create Repository**

**Step 8: Link Local Repository to GitHub**

```
git remote add origin https://github.com/username/repository-name.git
```

This connects your local project to the GitHub repository.

**Step 9: Push Code to GitHub**

```
git branch -M main
git push -u origin main
```

This uploads your files to GitHub.

**Step 10: Enable GitHub Pages**

- Go to your repository on GitHub
- Click **Settings → Pages**
- Under **Source**:
    - Select **main branch**
    - Select **/root folder**
- Click **Save**

**Step 11: Access Hosted Website**

After deployment, open:

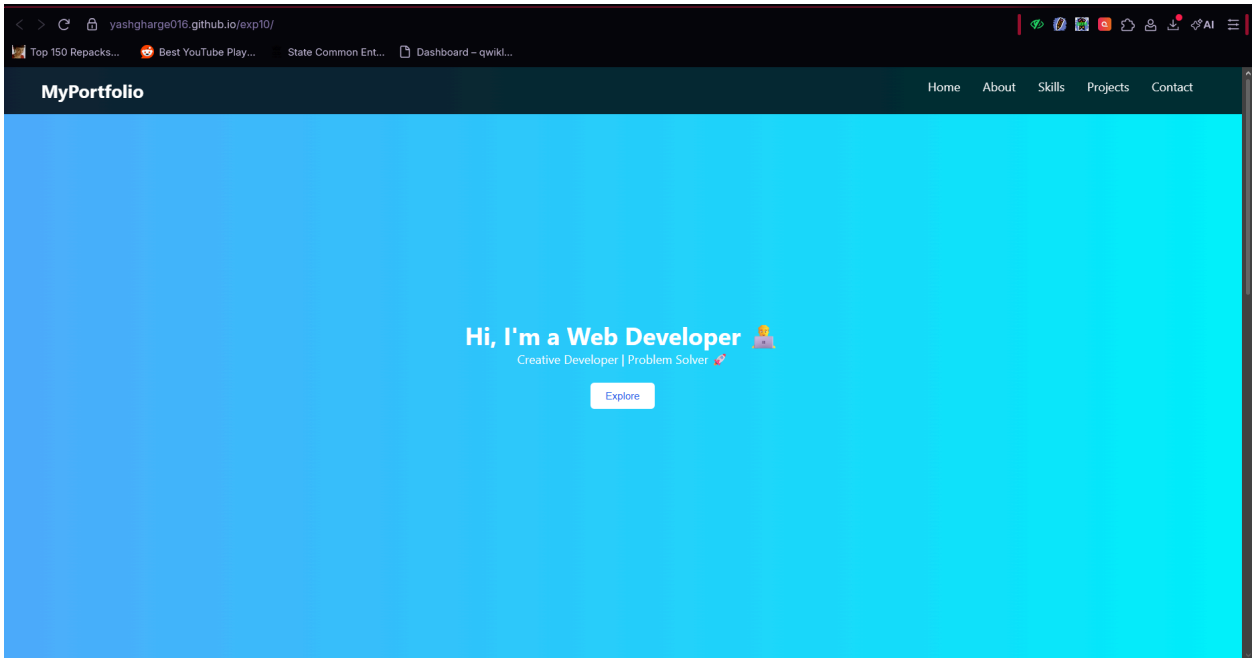
https://username.github.io/repository-name/

**Step 12: Update Website**

Whenever changes are made, use:

```
git add .
git commit -m "Updated website"
git push
```

The website will automatically update.




**Student Task:**

**1)Upload All Experiments to GitHub Using Git Commands**

Upload all completed experiments to a single GitHub repository using Git commands and organize them into separate folders.

**Steps (Using Git Commands)**

1. **Create a main folder for all experiments:**

web-programming-lab/

2. **Inside the folder, create subfolders:**

exp1/

exp2/

exp3/

...

exp10/

3. **Open terminal inside the main folder.**

**Initialize Git Repository**

git init

**Add All Files**

git add .

**Commit the Files**

git commit -m "Added all experiments"

**Connect to GitHub Repository**

git remote add origin https://github.com/username/web-programming-lab.git

**Push to GitHub**

git branch -M main

git push -u origin main

**Update Repository**

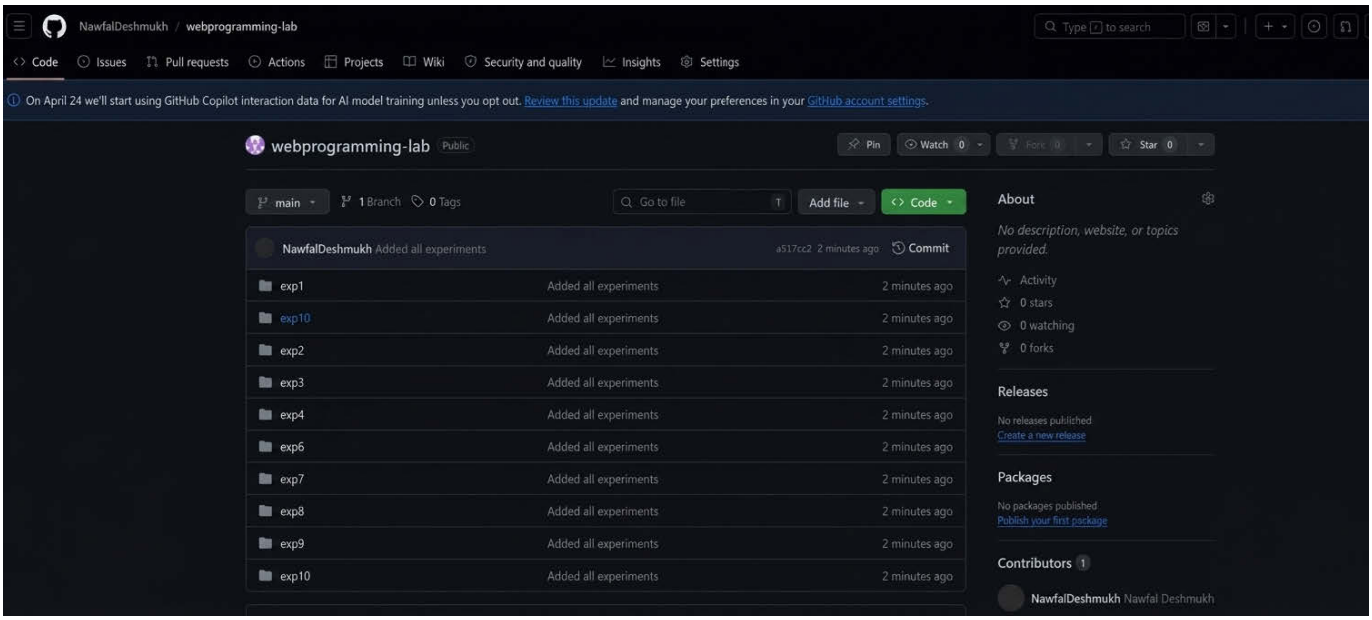
git add .

git commit -m "Updated experiments"

git push

```
Yash Gharge@LAPTOP-OQ7E0PCI MINGW64 ~/Desktop/wp experiments (main)
$ git push -u origin main
Counting objects: 100% (52/52), done.
Delta compression using up to 12 threads
Compressing objects: 100% (51/51), done.
Writing objects: 100% (52/52), 19.97 KiB | 352.00 KiB/s, done.
Total 52 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/YashGharge016/webprogramming-lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

Yash Gharge@LAPTOP-OQ7E0PCI MINGW64 ~/Desktop/wp experiments (main)
$
```


NawfalDeshmukh / webprogramming-lab
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. Review this update and manage your preferences in your GitHub account settings.
webprogramming-lab
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
T
Add file
Code
NawfalDeshmukh Added all experiments
a517cc2 2 minutes ago
Commit
exp1 Added all experiments 2 minutes ago
exp10 Added all experiments 2 minutes ago
exp2 Added all experiments 2 minutes ago
exp3 Added all experiments 2 minutes ago
exp4 Added all experiments 2 minutes ago
exp6 Added all experiments 2 minutes ago
exp7 Added all experiments 2 minutes ago
exp8 Added all experiments 2 minutes ago
exp9 Added all experiments 2 minutes ago
exp10 Added all experiments 2 minutes ago
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
NawfalDeshmukh Nawfal Deshmukh